Full Stack Development with MERN

Project Documentation

Online Complaint Registration and Management System

1. Introduction

Project Title

Online Complaint Registration and Management System

Team Members

Name	Role
Thumati Manasa	**Team Member**
Vemula Lalitha	**Member**
Pravallika Bolla	**Member**
Tanishka Dokiparthi	**Member**
Katikala Mohanakrishna	**Team Lead**

2. Project Overview

Purpose

The Online Complaint Registration and Management System is a web-based platform designed to digitize and streamline the process of lodging, tracking, and resolving public or organizational complaints. The system replaces manual, paper-based complaint handling with a centralized digital workflow that connects complainants (users), support staff (agents), and administrators on a single platform. Its primary goal is to improve transparency, accountability, and turnaround time in complaint resolution by providing structured complaint routing, real-time status tracking, and direct communication between users and the staff handling their complaints.

Features

- Secure user registration and login with role-based access control (User, Agent, Administrator).
- Complaint registration with category, priority, location, and file attachment support.
- Automatic and manual routing/assignment of complaints to support agents.
- Real-time complaint status tracking (Pending → Assigned → In Progress → Resolved/Rejected → Closed).
- Complete status-change history maintained for every complaint for auditability.
- In-app messaging thread between the complainant and the assigned agent for each complaint.
- Feedback and rating submission by users once a complaint is resolved.
- Notification system to alert users and agents of status changes, assignments, and new messages.
- Administrator dashboard for user management, role assignment, and complaint oversight.
- Analytics dashboard summarizing complaint volume, category distribution, and resolution metrics.
- Password reset and account recovery workflow.

3. Architecture

Frontend

The frontend is built using React.js and follows a component-based architecture. The user interface is organized into reusable components (forms, cards, tables, navigation) and page-level views (Dashboard, Complaint List, Complaint Detail, Profile, Admin Panel). Client-side routing is used to navigate between views without full page reloads, and centralized state (authenticated user, role, and session token) is managed through React Context so that role-based UI rendering and protected routes can be implemented consistently across the application. The frontend communicates with the backend exclusively through RESTful API calls made with Axios, and displays real-time UI updates such as unread notification counts and live status badges.

Backend

The backend is implemented using Node.js with the Express.js framework, exposing a REST API consumed by the React frontend. The backend is organized into routing, middleware, and controller layers: routes define the available endpoints, middleware handles authentication, authorization, and input validation, and controllers contain the business logic for each feature (complaint lifecycle management, user management, notifications, and messaging). This layered structure keeps request handling, security checks, and business logic separated and independently maintainable.

Database

MongoDB is used as the primary data store, with Mongoose as the Object Data Modeling (ODM) library to define schemas and enforce data structure at the application level. The core collections are Users, Complaints, StatusHistory, Messages, and Notifications. Complaints reference the creating user and, once assigned, the responsible agent. Every status transition on a complaint is written to the StatusHistory collection, providing a complete audit trail. The Entity-Relationship diagram below illustrates these collections and their relationships.

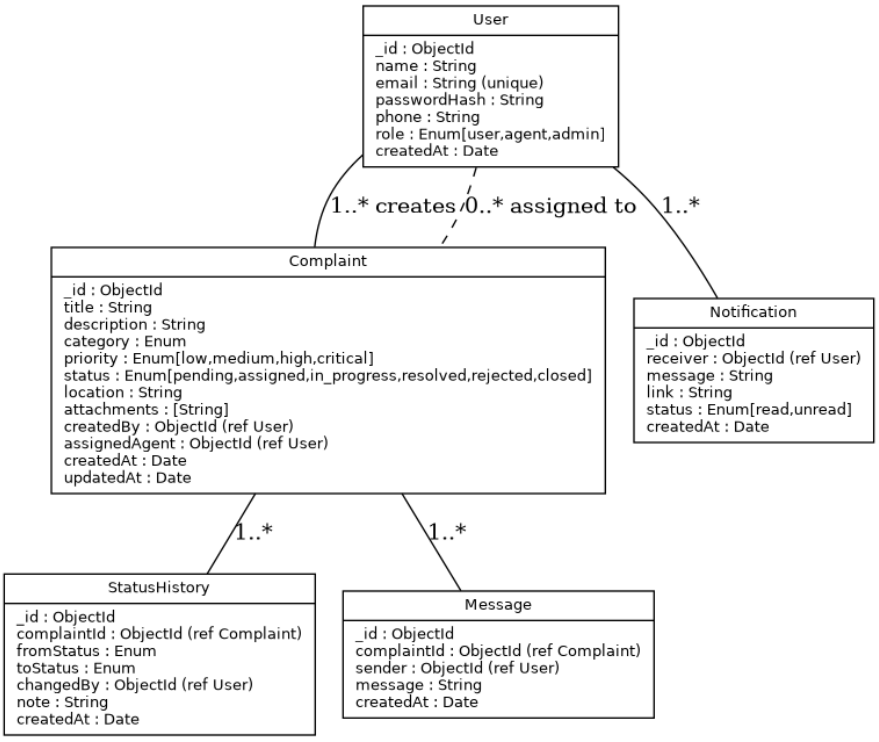


Figure 3.1: Entity-Relationship Diagram of the Complaint Management Database

System Architecture Diagram

The following diagram shows the three-tier architecture of the system: the React client layer, the Node.js/Express application layer, and the MongoDB data layer.

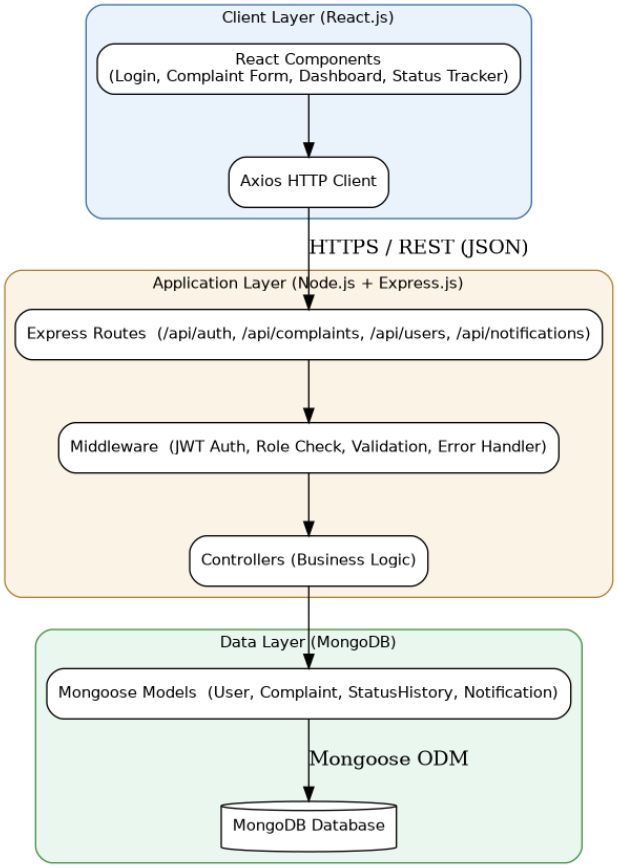


Figure 3.2: Three-Tier System Architecture

Complaint Submission Flow

The sequence diagram below illustrates the flow of a typical complaint-submission request from the client through authentication, business logic, persistence, and the resulting notification.

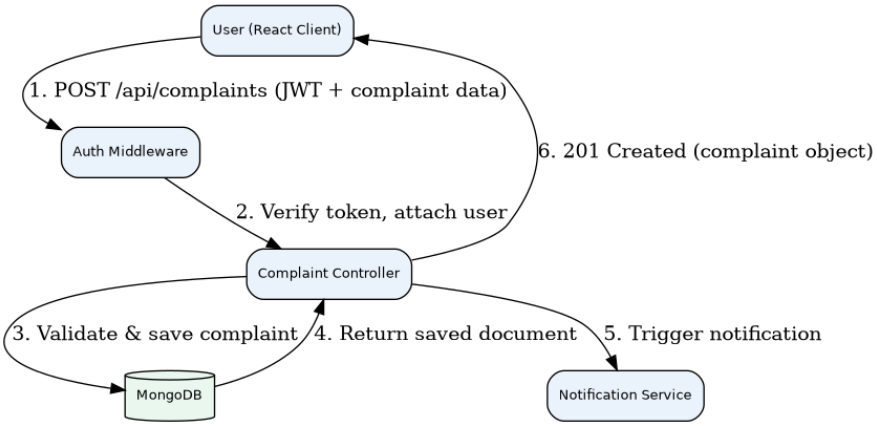


Figure 3.3: Sequence Diagram — Complaint Submission

Use Case Diagram

The use case diagram below summarizes the interactions available to each of the three user roles supported by the system.

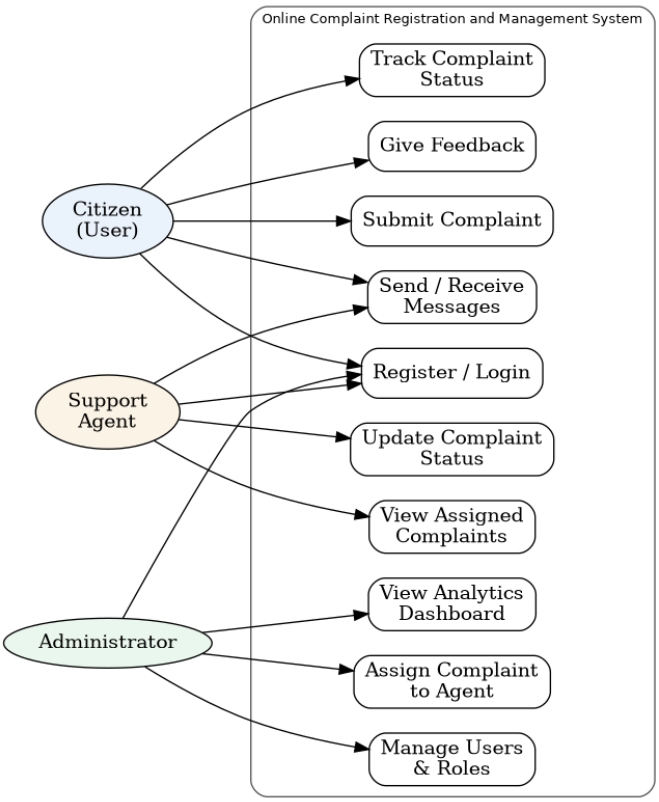


Figure 3.4: Use Case Diagram

4. Setup Instructions

Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local installation or a MongoDB Atlas cloud cluster)
- Git
- A code editor such as Visual Studio Code
- Postman (recommended, for testing API endpoints)

Installation

- Clone the repository: git clone <repository-url>
- Navigate to the server directory and install backend dependencies: cd server && npm install
- Navigate to the client directory and install frontend dependencies: cd client && npm install
- Create a .env file in the server directory and configure the environment variables listed below.
- Create a .env file in the client directory with the API base URL, e.g. REACT_APP_API_URL=http://localhost:5000/api.
- Start MongoDB (locally, or ensure the Atlas connection string is reachable).

Environment Variables

Variable	Description	Example
PORT	Port on which the Express server runs	5000
MONGO_URI	MongoDB connection string	mongodb://localhost:27017/complaint_system
JWT_SECRET	Secret key used to sign JWT tokens	--
JWT_EXPIRES_IN	Token validity duration	7d
EMAIL_SERVICE_USER	Email account used for notification emails	---
EMAIL_SERVICE_PASS	Email account app password	---

5. Folder Structure

Client

The React frontend follows a standard feature-oriented structure:

- client/src/components — reusable UI components (forms, cards, navigation bar, sidebar, complaint status badge).
- client/src/pages — top-level page views (Login, Register, Dashboard, Complaint List, Complaint Detail, New Complaint, Profile, Admin Panel, Analytics).
- client/src/context — React Context providers for authentication state and global user session.
- client/src/services — Axios API service modules (authService, complaintService, notificationService, userService).
- client/src/hooks — custom React hooks.
- client/src/routes — route definitions and protected-route wrappers based on user role.
- client/src/App.js — root component and route configuration.

Server

The Node.js/Express backend follows a layered MVC-style structure:

- server/config — database connection and environment configuration.
- server/models — Mongoose schemas (User, Complaint, StatusHistory, Message, Notification).
- server/routes — Express route definitions grouped by resource (authRoutes, complaintRoutes, userRoutes, notificationRoutes).
- server/controllers — business logic for each resource.
- server/middleware — authentication (JWT verification), role-based authorization, and input validation.
- server/utils — helper functions (e.g., token generation, email sending).
- server/server.js — application entry point that initializes Express, connects to MongoDB, and mounts the routes.

6. Running the Application

Once dependencies are installed and environment variables are configured, the frontend and backend are run as two separate processes during development:

- Backend: from the server directory, run npm start (or npm run dev if a development script with auto-reload, e.g. nodemon, is configured). The API server starts on the port defined by PORT (default 5000).
- Frontend: from the client directory, run npm start. The React development server starts on http://localhost:3000 and proxies API requests to the backend.

Both processes must be running simultaneously for the application to function, since the React client depends on the Express API for all data operations.

7. API Documentation

All endpoints are prefixed with /api and, unless stated otherwise, require a valid JWT bearer token in the Authorization header. The table below summarizes the primary endpoints exposed by the backend.

Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account (default role: user).
POST	/api/auth/login	Authenticate a user and return a JWT token.
POST	/api/auth/forgot-password	Send a password-reset link/token to the user's registered email.
POST	/api/auth/reset-password	Reset the account password using a valid reset token.
GET	/api/auth/me	Return the profile of the currently authenticated user.

Complaint Endpoints

Method	Endpoint	Description
POST	/api/complaints	Create a new complaint (authenticated users).
GET	/api/complaints	List complaints — scoped to the requester's own complaints, or all complaints for agents/admins.
GET	/api/complaints/:id	Retrieve full details of a single complaint, including status history.
PUT	/api/complaints/:id	Update complaint details (owner or admin).
PUT	/api/complaints/:id/assign	Assign a complaint to a support agent (admin only).
PUT	/api/complaints/:id/status	Update the status of a complaint (assigned agent or admin).
POST	/api/complaints/:id/feedback	Submit a rating and comment after a complaint is resolved.

Messaging & Notification Endpoints

Method	Endpoint	Description
GET	/api/complaints/:id/messages	Retrieve the message thread for a complaint.
POST	/api/complaints/:id/messages	Send a new message on a complaint thread.
GET	/api/notifications	Retrieve notifications for the authenticated user.
PUT	/api/notifications/:id/read	Mark a notification as read.

Administration Endpoints

Method	Endpoint	Description
GET	/api/users	List all registered users (admin only).
PUT	/api/users/:id/role	Change a user's role, e.g. promote to agent (admin only).
GET	/api/analytics	Retrieve aggregated statistics for the analytics dashboard (admin only).

Example Request / Response

POST /api/complaints

Request body: { "title": "Streetlight not working", "description": "The streetlight near Block C has been non-functional for a week.", "category": "electricity", "priority": "medium", "location": "Block C, Main Road" }

Response (201 Created): { "_id": "651f...", "title": "Streetlight not working", "status": "pending", "createdBy": "64ab...", "createdAt": "2026-07-10T10:15:00.000Z" }

Note: Exact request/response payloads should be verified against the implemented controllers and adjusted if field names differ.

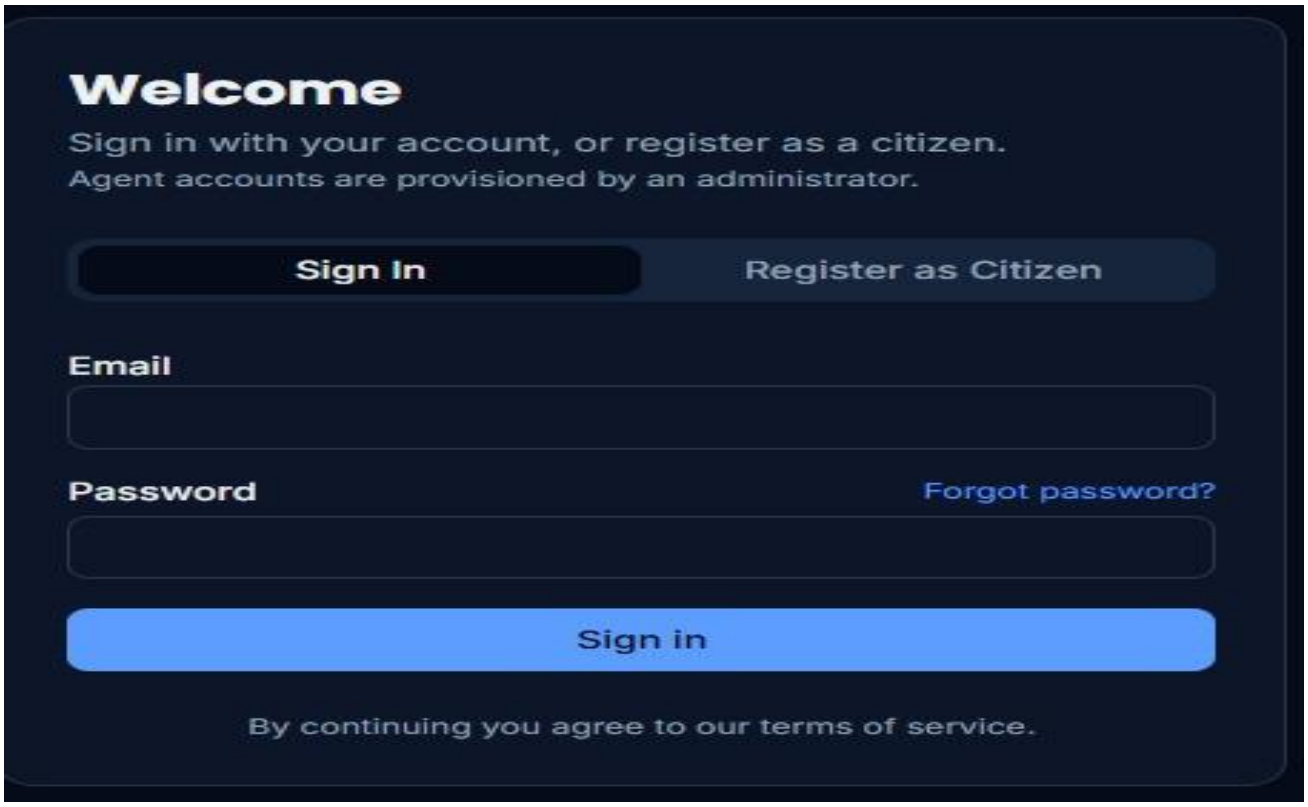
8. Authentication

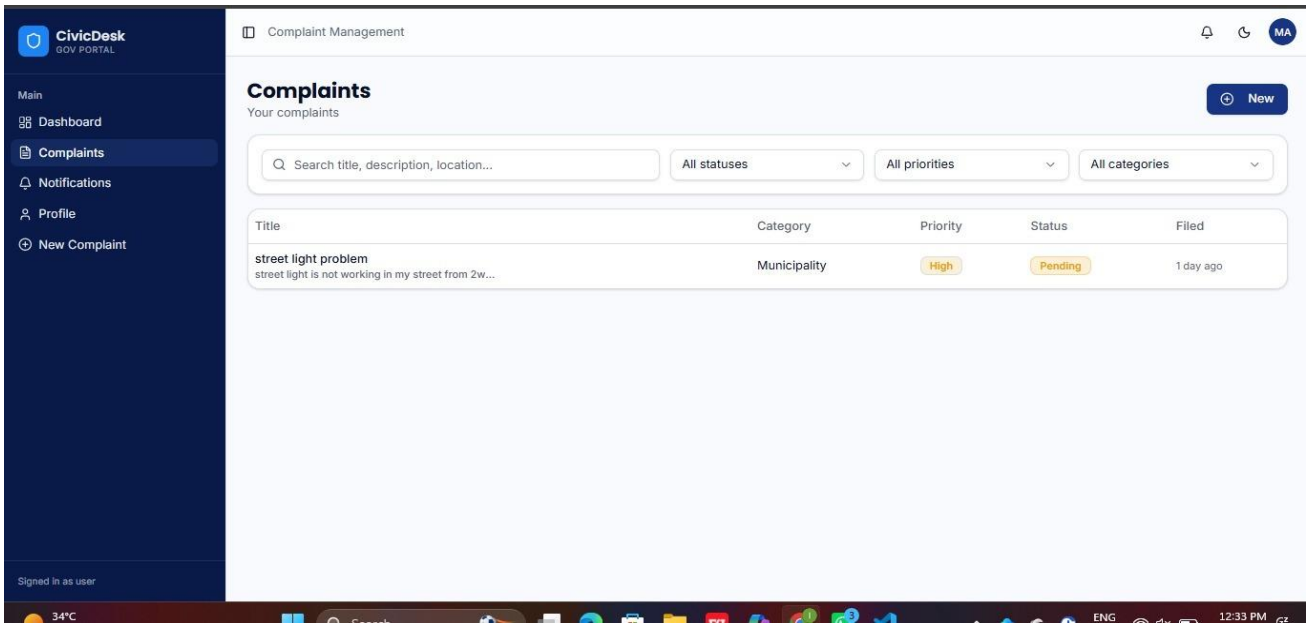
The system uses stateless, token-based authentication implemented with JSON Web Tokens (JWT):

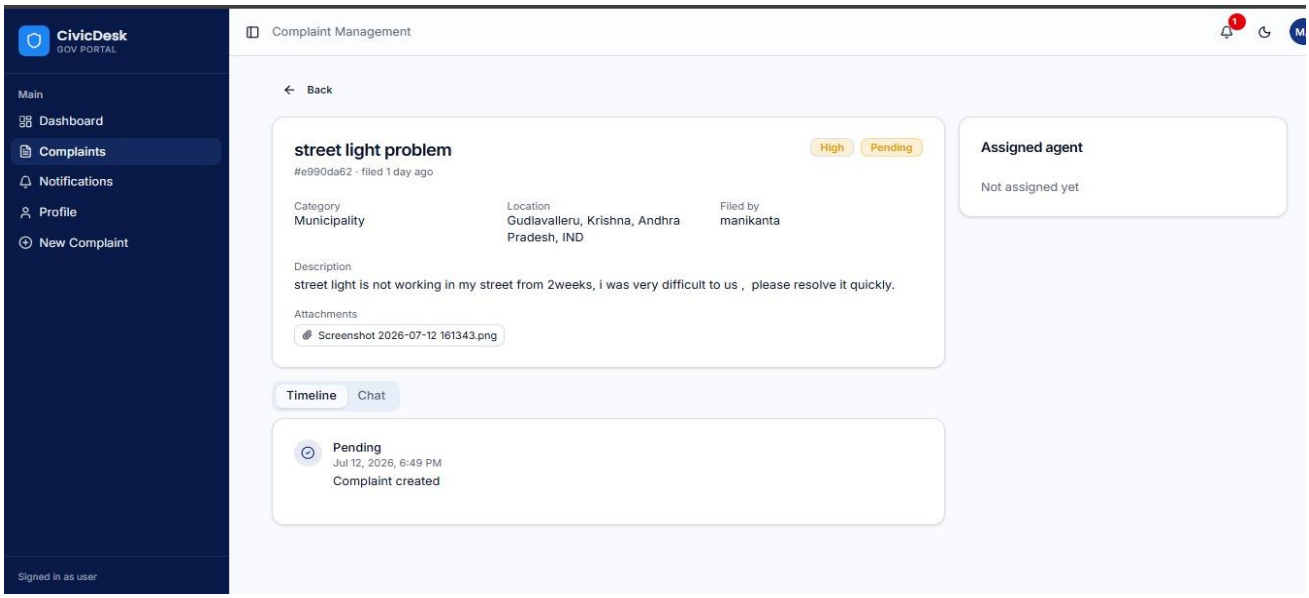
- On registration, user passwords are hashed using bcrypt before being stored; plain-text passwords are never persisted.
- On login, credentials are verified against the stored hash, and upon success the server issues a signed JWT containing the user's ID and role.
- The client stores the token and attaches it as a Bearer token in the Authorization header of subsequent API requests.
- An authentication middleware on the server verifies the token's signature and expiry on every protected route, attaching the decoded user to the request object.
- A role-based authorization middleware restricts specific routes (e.g., assigning complaints, managing users) to Agent or Administrator roles only.
- Password reset uses a time-limited token sent to the user's registered email, allowing the user to set a new password without exposing the old one.

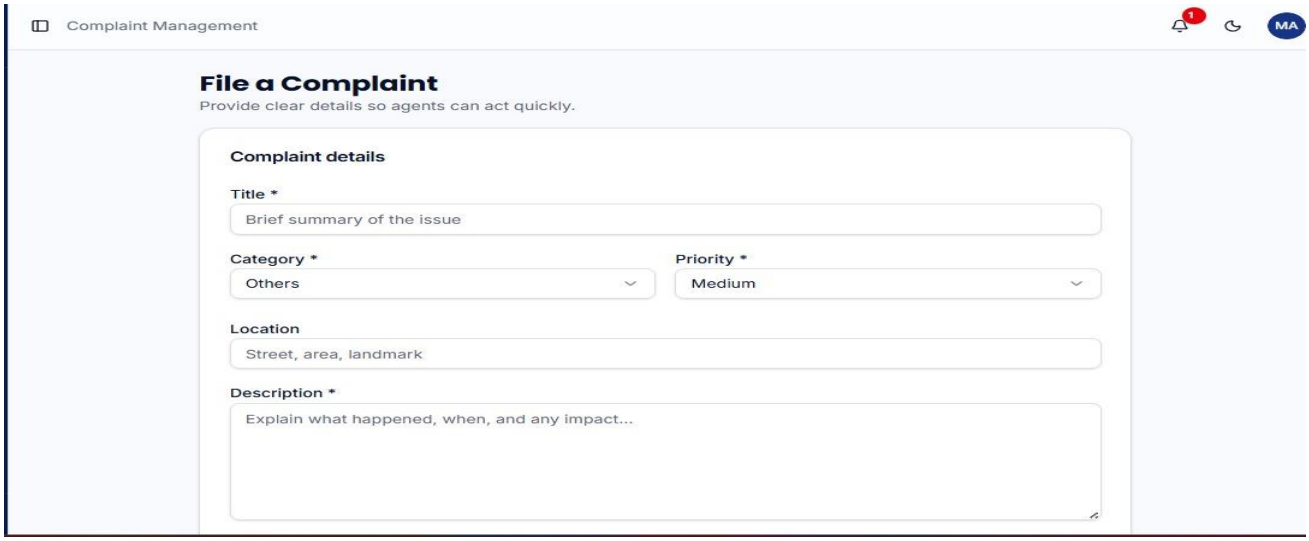
9. User Interface

The interface is organized around three role-specific experiences layered over a shared design system: a citizen-facing view for submitting and tracking complaints, an agent view for managing assigned complaints, and an administrator view for platform-wide oversight.




CivicDesk
GOV PORTAL
Complaint Management
MA
Main
Dashboard
Complaints
Notifications
Profile
New Complaint
Complaints
Your complaints
New
Search title, description, location...
All statuses
All priorities
All categories
Title
Category
Priority
Status
Filed
street light problem
street light is not working in my street from 2w...
Municipality
High
Pending
1 day ago
Signed in as user
34°C
ENG
12:33 PM


CivicDesk
GOV PORTAL
Complaint Management
Main
Dashboard
Complaints
Notifications
Profile
New Complaint
Back
street light problem
High
Pending
#e990da62 · filed 1 day ago
Category
Municipality
Location
Gudlavalleru, Krishna, Andhra Pradesh, IND
Filed by
manikanta
Description
street light is not working in my street from 2weeks, i was very difficult to us , please resolve it quickly.
Attachments
Screenshot 2026-07-12 161343.png
Assigned agent
Not assigned yet
Timeline
Chat
Pending
Jul 12, 2026, 6:49 PM
Complaint created
Signed in as user


Complaint Management
MA
File a Complaint
Provide clear details so agents can act quickly.
Complaint details
Title *
Brief summary of the issue
Category *
Others
Priority *
Medium
Location
Street, area, landmark
Description *
Explain what happened, when, and any impact...

10. Testing

API endpoints were tested manually using Postman to verify correct request/response behavior, status codes, and role-based access restrictions across the authentication, complaint, messaging, and notification routes. Frontend functionality was verified through manual functional testing of each user flow (registration, login, complaint submission, status tracking, messaging, and feedback) across the three roles. An automated unit/integration test suite (e.g., using Jest and React Testing Library on the client, and Jest/Supertest on the server) was not part of the described implementation and is recommended as a future addition rather than assumed to exist.

11. Known Issues

- Notifications are delivered by polling/refresh rather than a persistent real-time channel (e.g., WebSockets), which may introduce a short delay before updates appear.
- File attachments on complaints are limited by server-side upload size restrictions; large media files (e.g., videos) may fail to upload.
- No automated test suite is currently in place, so regressions must be caught through manual testing.
- Complaint listing does not yet implement server-side pagination, which may affect performance as complaint volume grows.
- Concurrent status updates to the same complaint by multiple agents are not explicitly conflict-resolved.

This list reflects limitations typical of the described scope and should be reviewed and updated against the actual implementation before final submission.

12. Future Enhancements

- Integrate WebSockets (e.g., Socket.IO) for real-time notifications and live chat updates.
- Add SMS and email alerts for critical status changes, in addition to in-app notifications.
- Expand the analytics dashboard with visual charts for complaint trends, category breakdowns, and average resolution time.
- Introduce AI-assisted complaint categorization and priority suggestion based on the description text.
- Develop a companion mobile application for Android and iOS.
- Add multi-language support to improve accessibility for a wider user base.
- Implement server-side pagination, filtering, and search for large complaint datasets.
- Add an automated test suite (unit, integration, and end-to-end tests) to improve reliability.